

POLITECNICO GRAN COLOMBIANO

Facultad de Ingenieria, Diseno e Innovacion

Ingenieria de Software

Analisis en la implementacion de la arquitectura de software orientada a servicios

Entrega Final

Modulo: Arquitectura del Software

Docente: Isabel Andrea Mahecha Nieto

Integrantes:

Cristian Ricardo Rojas

Juan Jose Gonzalez Quintero

Luigy Yesid Nieves Wintaco

Yeison Orlando Hernandez Tibaquira

Bogota D.C., Colombia - 2026

1. Introduccion

En la actualidad la transformación digital ha incrementado significativamente el intercambio de información a través de internet, lo que también ha generado un aumento en los riesgos asociados a la seguridad de los datos personales. Las filtraciones de información, el robo de credenciales y la exposición de correos electrónicos en las bases de datos comprometidas representan una amenaza constante para los usuarios y las organizaciones. Frente a la problemática, surge la necesidad de desarrollar soluciones tecnológicas que permitan detectar vulnerabilidades de forma oportuna y orientar a los usuarios en la protección de su información.

En este contexto, el proyecto CiberEscudo se plantea como una plataforma web enfocada en el monitoreo y prevención de riesgos de ciberseguridad personal, permitiendo verificar si cuentas de correo electrónico o contraseñas han sido comprometidas en brechas de seguridad conocidas, mediante la integración con servicios externos especializados.

Para responder eficientemente a esta necesidad, se propone la implementación de una arquitectura orientada a servicios (SOA), la cual permite dividir el sistema en servicios independientes, reutilizables y escalables, facilitando la comunicación entre componentes mediante protocolos estándar. Dentro de este enfoque, se selecciona el modelo REST como mecanismo principal de intercambio de información, debido a su simplicidad, flexibilidad y compatibilidad con aplicaciones web modernas.

El presente documento expone la propuesta del servicio REST para CiberEscudo y presenta las conclusiones finales del análisis arquitectónico, evaluando el cumplimiento de los objetivos planteados, los criterios para seleccionar otros patrones de arquitectura y las ventajas que ofrecen las arquitecturas orientadas a servicios en el desarrollo de soluciones modernas, seguras y eficientes.

La arquitectura de software orientada a servicios (SOA, por sus siglas en inglés) constituye un paradigma esencial en el desarrollo de sistemas empresariales modernos. Su fortaleza radica en la capacidad de descomponer una aplicación en servicios independientes, reutilizables e interoperables, lo que facilita la escalabilidad, el mantenimiento y la integración con terceros (Amazon Web Services, 2022).

El presente documento corresponde a la primera entrega del trabajo grupal de la asignatura Arquitectura del Software. En él se desarrolla el análisis de una solución de software orientada a servicios, aplicada a una problemática real y recurrente en el ámbito de la ciberseguridad ciudadana. Para ello, se seleccionó la API pública Have I Been Pwned (HIBP), perteneciente a la categoría Security del repositorio de APIs públicas en GitHub (public-apis, 2024).

El documento aborda la descripción de la idea de aplicación, sus requerimientos funcionales y no funcionales, el estilo arquitectónico seleccionado y los tres patrones de arquitectura propuestos, con su respectiva argumentación.

2. Descripción de la aplicación

2.1 API seleccionada

La API seleccionada es Have I Been Pwned (HIBP), disponible en <https://haveibeenpwned.com/API/v3>. Esta API pertenece a la categoría Security del listado de APIs públicas en GitHub. Permite consultar si un correo electrónico, nombre de usuario o contraseña han sido expuestos en filtraciones de datos (data breaches) documentadas a nivel mundial. Para las verificaciones de contraseña, utiliza el modelo de anonimización k-Anonymity, que garantiza que ninguna contraseña viaja en texto plano por la red: únicamente se envían los primeros cinco caracteres del hash SHA-1, protegiendo así la privacidad del usuario (Hunt, 2018).

2.2 Nombre e idea de la aplicación

La aplicación se denomina CiberEscudo. Es una plataforma web de monitoreo y concientización en ciberseguridad personal, orientada a ciudadanos colombianos que utilizan servicios digitales cotidianos —banca en línea, redes sociales, correo electrónico, plataformas de comercio electrónico— pero que no cuentan con conocimientos técnicos en seguridad informática.

CiberEscudo permite a sus usuarios registrar uno o más correos electrónicos para ser monitoreados de forma continua. Cuando el sistema detecta que alguno de esos correos aparece en una nueva filtración de datos, envía una alerta inmediata y proporciona una guía de acción paso a paso, redactada en lenguaje claro y sin jerga técnica, indicando qué servicios fueron comprometidos y qué acciones concretas debe tomar el usuario (cambiar contraseña, activar doble factor de autenticación, revisar movimientos bancarios, etc.).

2.3 ¿Por qué es una idea novedosa?

Si bien existen herramientas similares a nivel global —como la página oficial de HIBP o el monitor de contraseñas de Google— ninguna está contextualizada para el entorno colombiano ni latinoamericano. CiberEscudo se diferencia en los siguientes aspectos:

- Localización completa al español con lenguaje accesible para usuarios no técnicos.
- Guías de acción específicas para servicios colombianos ampliamente usados: Bancolombia, Nequi, Rappi, Claro, EPM, entre otros.
- Panel de historial personal que consolida todas las alertas recibidas, permitiendo al usuario tener trazabilidad de su exposición digital a lo largo del tiempo.

- Módulo educativo breve integrado en cada alerta, que explica en qué consiste el tipo de brecha detectada y cómo prevenirla en el futuro.
- Diseño centrado en la accesibilidad, compatible con lectores de pantalla y con opciones de contraste alto, pensado también para adultos mayores.

2.4 ¿Para qué sirve y qué necesidades cubre?

CiberEscudo resuelve el problema de visibilidad y acción frente a brechas de seguridad. La mayoría de personas afectadas por filtraciones de datos no se entera de que sus credenciales están comprometidas hasta que sufren un fraude o un acceso no autorizado a sus cuentas. Esta demora en la detección incrementa significativamente el daño potencial.

La aplicación cubre las siguientes necesidades concretas:

- Detección temprana: el usuario es notificado en el menor tiempo posible tras la incorporación de una brecha a la base de datos de HIBP.
- Orientación post-brecha: muchos usuarios no saben qué hacer al enterarse de que sus datos fueron expuestos; CiberEscudo les proporciona pasos concretos y priorizados.
- Verificación de contraseñas: permite comprobar si una contraseña específica ha sido filtrada, sin comprometer su privacidad gracias al modelo k-Anonymity.
- Educación continua: contribuye a elevar el nivel de conciencia en ciberseguridad de la población colombiana, reduciendo la reutilización de contraseñas y otras malas prácticas.

2.5 Población objetivo

La aplicación está dirigida principalmente a adultos colombianos de entre 20 y 65 años que utilizan servicios digitales de manera frecuente pero no tienen formación técnica en seguridad informática. Dentro de este segmento se destacan: empleados de pequeñas y medianas empresas (pymes) que no cuentan con un departamento de TI, adultos mayores que han adoptado la banca digital, estudiantes universitarios y emprendedores que gestionan su identidad digital en múltiples plataformas. La aplicación también es relevante para departamentos de recursos humanos y áreas administrativas que manejan información sensible de terceros.

3. Descripción detallada de la situación problema

En Colombia y en gran parte de Latinoamérica, los ciudadanos que utilizan servicios digitales —banca en línea, correo electrónico, redes sociales, plataformas de comercio electrónico— se enfrentan a una problemática recurrente y silenciosa: la exposición de sus datos personales a través de filtraciones de seguridad (data breaches) sin que tengan conocimiento oportuno de ello ni orientación concreta sobre cómo actuar.

Según el informe de IBM Security (2023), el tiempo promedio global para identificar y contener una brecha de datos es de 277 días. Durante ese lapso, las credenciales comprometidas circulan en mercados clandestinos de la dark web, siendo utilizadas para fraudes bancarios, robo de identidad y accesos no autorizados a cuentas. La mayoría de los usuarios afectados solo se entera de la situación cuando sufre un daño tangible, como un cargo no reconocido o el bloqueo de su cuenta.

La situación se agrava por la ausencia de herramientas accesibles y contextualizadas para el entorno colombiano. Las plataformas existentes, como el sitio web oficial de Have I Been Pwned (HIBP) o el monitor de contraseñas de Google, están diseñadas en inglés y carecen de orientación específica para los servicios digitales más utilizados en Colombia (Nequi, Bancolombia, Rappi, Claro, EPM, entre otros). Esto genera una brecha de accesibilidad que margina precisamente a los usuarios más vulnerables: adultos mayores, personas con bajo nivel de alfabetización digital y trabajadores de pymes sin soporte de TI.

Desde la perspectiva de la arquitectura de software, la situación problema se puede enunciar así: ¿cómo diseñar un sistema orientado a servicios que integre una API externa de seguridad (HIBP), procese eventos de brecha de forma asíncrona, almacene el historial de exposición del usuario, y entregue notificaciones y guías de acción en tiempo real, garantizando disponibilidad, escalabilidad y cumplimiento de la normativa colombiana de protección de datos (Ley 1581 de 2012)?

Esta es la situación problema que CiberEscudo busca resolver: la falta de visibilidad, detección temprana y orientación post-brecha para el ciudadano colombiano en materia de seguridad de sus datos personales en entornos digitales.

4. Referentes

2.1 Patrones de arquitectura

- Erl, T. (2009). SOA Design Patterns. Prentice Hall. — Fuente principal para los patrones Service Registry, Publish-Subscribe y Service Façade aplicados en el diseño de CiberEscudo.
- Richardson, C. (2018). Microservices Patterns: With Examples in Java. Manning Publications. — Proporciona el patrón API Gateway y el patrón Saga para la gestión de transacciones distribuidas.

- Richards, M., & Ford, N. (2020). *Fundamentals of Software Architecture: An Engineering Approach*. O'Reilly Media. — Marco de referencia para la selección del estilo arquitectónico y los atributos de calidad.

2.2 Libros y artículos de investigación relacionados

- IBM Security. (2023). *Cost of a Data Breach Report 2023*. IBM Corporation. — Datos empíricos sobre el tiempo de detección y costo de las brechas de seguridad a nivel global.
- Shostack, A. (2014). *Threat Modeling: Designing for Security*. Wiley. — Marco de modelado de amenazas aplicado al análisis de riesgos en CiberEscudo.
- Caivano, D., Fernández-Alemán, J. L., & Toval, A. (2019). Personal data protection in social networking sites: A systematic review. *Computers & Security*, 77, 382–400. — Evidencia académica sobre la exposición de datos personales en plataformas digitales.
- Superintendencia de Industria y Comercio (SIC). (2022). *Informe de actividad de datos personales en Colombia 2022*. — Contexto normativo y estadístico del problema en el entorno colombiano.

2.3 Informes técnicos de herramientas relacionadas

- Hunt, T. (2018). *Have I Been Pwned: Passwords API v3 Documentation*. <https://haveibeenpwned.com/API/v3> — Especificación técnica de la API utilizada por CiberEscudo, incluyendo el modelo k-Anonymity.
- Amazon Web Services. (2022). *¿Qué es la arquitectura orientada a servicios?* <https://aws.amazon.com/es/what-is/service-oriented-architecture/> — Referente conceptual sobre los componentes SOA.
- OWASP Foundation. (2021). *OWASP Top Ten 2021*. <https://owasp.org/www-project-top-ten/> — Marco de referencia para la identificación de vulnerabilidades en sistemas web, aplicado al diseño seguro de CiberEscudo.

5. Justificación

La protección de datos personales en entornos digitales constituye uno de los retos más urgentes de la era tecnológica actual. Según el informe de IBM Security (2023), en ese año se registraron filtraciones que afectaron a más de 2.600 millones de registros a nivel mundial, con un costo promedio por brecha de 4,45 millones de dólares. En el contexto colombiano, la Superintendencia de Industria y Comercio ha reportado un incremento sostenido en las denuncias relacionadas con el uso indebido de datos personales, lo que evidencia una brecha significativa entre la frecuencia real de las filtraciones y el conocimiento que los ciudadanos tienen sobre su propia exposición digital.

Analizar esta situación resulta fundamental porque permite comprender de qué manera una arquitectura de software orientada a servicios puede dar respuesta concreta a un problema real y multidimensional: la ausencia de visibilidad, detección oportuna y orientación post-brecha para el ciudadano colombiano. El análisis propuesto con CiberEscudo permite identificar cómo descomponer el problema en servicios independientes —monitoreo, notificación, verificación y guía de acción—, cada uno con responsabilidades claras y contratos de comunicación bien definidos, aplicando los principios SOA de bajo acoplamiento, alta cohesión y reutilización de servicios.

Este análisis puede servir como modelo de referencia para el diseño de soluciones de ciberseguridad ciudadana en otros contextos latinoamericanos, y como caso de estudio sobre la aplicación práctica de los principios SOA en dominios de alta sensibilidad, donde la disponibilidad, la seguridad y los atributos de calidad no funcionales son requisitos críticos e innegociables. Comprender cómo un sistema de estas características debe estructurar sus servicios, gestionar sus proveedores y documentar sus patrones arquitectónicos es una competencia esencial para el ingeniero de software contemporáneo.

6. Objetivos

6.1 Objetivo general

Implementar una solución basada en Arquitectura Orientada a Servicios (SOA) de tipo API de Have I Been Pwned (HIBP), para llevar a cabo el acceso y la consulta de las posibles filtraciones de datos asociadas a las direcciones de correo electrónico o credenciales de usuario, permitiendo así detectar posibles vulnerabilidades de seguridad y proteger la información.

6.2 Objetivos específicos

- Analizaremos los componentes clave de la Arquitectura Orientada a Servicios (SOA) y a identificar cómo cada uno de ellos contribuye a la solución que proponemos para integrar con la API de Have I Been Pwned.
- Diseñar e implementar un servicio que utilice la API de Have I Been Pwned, lo que permitirá hacer consultas sobre posibles brechas de seguridad relacionadas con las cuentas de usuario.
- Es importante definir cómo funciona el proveedor de servicios, asegurando que el sistema pueda ofrecer de manera eficiente y accesible el servicio de consulta sobre filtraciones para los usuarios.
- Crea un registro de servicios que te ayude a identificar y gestionar los servicios disponibles en la arquitectura del sistema.

- Aplicar patrones de diseño de SOA puede ser una excelente manera de mejorar la organización, la reutilización y la escalabilidad del sistema, especialmente en el contexto de la seguridad de la información.

7. Requerimientos funcionales

A continuación se presenta el listado de requerimientos funcionales de CiberEscudo, organizados por módulo de la aplicación:

A continuación se presenta el listado de requerimientos funcionales de CiberEscudo, organizados por módulo de la aplicación:

ID	Módulo	Descripción
RF-01	Gestión de acceso	El sistema debe permitir el acceso mediante un usuario y contraseña fijos definidos en la configuración del sistema. No se requiere registro de nuevos usuarios en el MVP.
RF-02	Gestión de correos	El sistema debe permitir al usuario agregar, editar y eliminar los correos electrónicos que desea monitorear.
RF-03	Monitoreo de correos	El sistema debe consultar la API HIBP y retornar la lista de brechas asociadas a cada correo registrado.
RF-04	Monitoreo de correos	El sistema debe actualizar el estado de monitoreo de cada correo al menos una vez por día de forma automática.
RF-05	Alertas	El sistema debe enviar una notificación por correo electrónico al usuario cuando uno de sus correos monitoreados aparezca en una nueva brecha.
RF-06	Alertas	El sistema debe mostrar en el panel de usuario un resumen de todas las alertas recibidas con fecha, nombre de la brecha y datos comprometidos.
RF-07	Verificación de contraseñas	El sistema debe permitir al usuario verificar si una contraseña específica ha sido expuesta, utilizando el modelo k-Anonymity de HIBP (solo los primeros 5 caracteres del hash SHA-1).

ID	Módulo	Descripción
RF-08	Guías de acción	El sistema debe generar automáticamente una guía de acción en lenguaje no técnico para cada tipo de brecha detectada, con pasos priorizados.
RF-09	Guías de acción	El sistema debe incluir en la guía referencias específicas a servicios colombianos cuando el tipo de brecha lo amerite.
RF-10	Historial	El sistema debe mantener un historial completo de consultas y alertas por usuario, consultable por rango de fechas.
RF-11	Módulo educativo	El sistema debe mostrar al usuario una explicación breve sobre el tipo de brecha detectada (contraseñas, emails, datos financieros, etc.) y cómo prevenirla.
RF-12	Reportes	El sistema debe permitir al usuario exportar su historial de alertas en formato PDF o CSV.

8. Requerimientos no funcionales

Los requerimientos no funcionales definen los atributos de calidad que debe cumplir la aplicación CiberEscudo.

Los requerimientos no funcionales definen los atributos de calidad que debe cumplir la aplicación CiberEscudo para garantizar su correcto funcionamiento en un entorno de producción real.

ID	Atributo de calidad	Descripción
RNF-01	Seguridad	Las contraseñas de cuentas externas ingresadas para verificación nunca se transmiten en texto plano ni se almacenan. Se aplica el modelo k-Anonymity: únicamente se envían los primeros 5 caracteres del hash SHA-1 a la API HIBP. Las credenciales de acceso al sistema (usuario y contraseña de la app) se definen como constantes en el archivo de configuración local.
RNF-02	Seguridad	En el entorno MVP local, la comunicación se realiza sobre HTTP en localhost. Las credenciales de acceso al sistema no se almacenan en base de datos: están definidas como variables fijas en config.py, lo que es suficiente y seguro para un entorno local de desarrollo.
RNF-03	Disponibilidad	En el contexto MVP local, el sistema debe mantenerse operativo durante las sesiones de uso del usuario en su máquina. Para versiones futuras en producción se define como meta una disponibilidad del 99 % mensual.
RNF-04	Rendimiento	El tiempo de respuesta de una consulta de verificación de correo no debe superar los 3 segundos en el entorno local, considerando el tiempo de respuesta de la API HIBP externa más el procesamiento local.
RNF-05	Escalabilidad	La arquitectura debe estar diseñada de forma modular (SOA) para que, en una versión futura productiva, pueda escalar añadiendo instancias de servicio sin rediseñar la lógica de negocio. En el MVP local la escalabilidad no es un requisito operativo inmediato.
RNF-06	Mantenibilidad	Cada módulo del sistema MVP debe tener responsabilidad única y ser modificable sin afectar a los demás. El código debe seguir los principios SOLID de manera básica, priorizando legibilidad y separación de responsabilidades acorde a un proyecto académico funcional.

ID	Atributo de calidad	Descripción
RNF-07	Usabilidad	La interfaz debe ser comprensible para usuarios sin conocimientos técnicos. El tiempo medio para completar una tarea principal (registrar un correo, consultar alerta) no debe superar los 2 minutos en usuarios nuevos.
RNF-08	Accesibilidad	La interfaz debe cumplir con las pautas WCAG 2.1 nivel AA: contraste de texto, navegación por teclado y compatibilidad con lectores de pantalla.
RNF-09	Interoperabilidad	El sistema debe consumir la API HIBP siguiendo su contrato de integración vigente (versión 3) y debe adaptarse ante cambios de versión sin interrupciones al usuario final.
RNF-10	Cumplimiento legal	El tratamiento de datos personales debe cumplir con la Ley 1581 de 2012 (Habeas Data) de Colombia. El sistema debe contar con política de privacidad visible y mecanismo de eliminación de cuenta con borrado de datos.

9. Estilo arquitectónico seleccionado

5.1 Estilo: Arquitectura Orientada a Servicios (SOA)

El estilo arquitectónico seleccionado para el desarrollo de CiberEscudo es la Arquitectura Orientada a Servicios (SOA, Service-Oriented Architecture). Este estilo organiza el sistema como un conjunto de servicios de software independientes, interoperables y reutilizables, que se comunican entre sí a través de interfaces bien definidas y protocolos estándar (Amazon Web Services, 2022).

5.2 Justificación de la selección

La elección de SOA responde a las características propias del problema que CiberEscudo busca resolver y a los atributos de calidad definidos en los requerimientos no funcionales. A continuación se argumenta cómo este estilo aporta al diseño de la aplicación:

a) Desacoplamiento e independencia de servicios. CiberEscudo puede descomponerse en servicios claramente diferenciados: Servicio de Autenticación, Servicio de Monitoreo de Correos, Servicio de Verificación de Contraseñas, Servicio de Notificaciones, Servicio de Guías de Acción e Historial. Cada uno puede desarrollarse, desplegarse y actualizarse de forma independiente, reduciendo el impacto de los cambios (RNF-06).

b) Integración nativa con APIs externas. SOA es el paradigma más adecuado cuando el sistema depende de servicios externos como HIBP. El Servicio de Monitoreo actúa como consumidor del servicio externo, y si HIBP cambia de versión, solo este servicio necesita adaptarse sin afectar al resto del sistema (RNF-09).

c) Escalabilidad horizontal. Dado que los servicios son independientes, es posible escalar únicamente los que estén bajo mayor demanda. Por ejemplo, si hay un incremento masivo de alertas tras una brecha de datos notable, solo el Servicio de Notificaciones requiere mayor capacidad computacional, sin necesidad de escalar toda la aplicación (RNF-05).

d) Reutilización de servicios. El Servicio de Guías de Acción puede ser consumido tanto por el panel web del usuario como por el sistema de notificaciones por correo, sin necesidad de duplicar lógica. Esto reduce la deuda técnica y mejora la consistencia del sistema.

e) Cumplimiento normativo. Al tener un Servicio de Gestión de Datos independiente, es más sencillo implementar y auditar el cumplimiento de la Ley 1581 de 2012, ya que toda operación sobre datos personales está centralizada en un único servicio con sus propios mecanismos de control (RNF-10).

10. Analisis de los componentes de la arquitectura orientada a servicios

A continuación se describen de manera detallada los principales componentes SOA de CiberEscudo, en perspectiva de la situación problema: la falta de visibilidad, detección oportuna y orientación post-brecha que enfrentan los ciudadanos colombianos ante filtraciones de datos personales.

2.1 Servicios

En CiberEscudo los servicios son los bloques funcionales del sistema. Cada uno encapsula una capacidad de negocio específica, es desplegable de forma independiente y se comunica con los demás a través de contratos de interfaz bien definidos. Se identifican servicios tanto para usuarios internos (comunicación entre componentes del sistema) como accesibles para todos (expuestos al cliente web a través del API Gateway).

2.1.1 Servicio de Autenticación y Gestión de Usuarios

Implementación del servicio: Gestiona el acceso al sistema mediante credenciales fijas definidas como constantes en config.py (APP_USER y APP_PASSWORD). El flujo es el siguiente: el usuario ingresa su usuario y contraseña en el formulario de login de Flask; el módulo los compara directamente con las constantes configuradas mediante una comparación de strings; si coinciden, establece session['logged_in'] = True en la sesión de Flask; todas las rutas protegidas verifican esta variable antes de mostrar contenido. No existe registro de usuarios, no hay base de datos para autenticación, no hay hashing de contraseñas. Es la forma más simple y directa de proteger un MVP local.

Accesible para todos: Sí. Este servicio expone endpoints públicos al cliente web a través del API Gateway.

Interfaz del servicio: POST /auth/login — recibe los campos 'usuario' y 'contraseña' del formulario HTML, los compara con APP_USER y APP_PASSWORD de config.py; si son correctos establece session['logged_in'] = True y redirige al panel; si son incorrectos muestra el formulario con mensaje de error. GET /auth/logout — limpia la sesión con session.clear() y redirige al login. El decorador @login_required (función auxiliar local) protege todas las rutas del sistema verificando session.get('logged_in').

2.1.2 Servicio de Monitoreo de Correos

Implementación del servicio: Es el servicio central de CiberEscudo y el núcleo de la solución a la situación problema. Gestiona los correos electrónicos registrados por el usuario y realiza consultas a la API HIBP cuando el usuario ejecuta la verificación desde la interfaz web, usando la librería requests de Python. Implementa un sleep() de 1,5 segundos entre solicitudes para no superar el rate limit de la API HIBP. En el MVP local no hay ejecución automática programada: el chequeo es disparado manualmente. Al detectar una brecha nueva, invoca

directamente las funciones `notify()` del módulo de Notificaciones y `register_alert()` del módulo de Historial mediante llamadas locales a función, sin necesidad de un bus de mensajes externo.

Accesible para todos: Sí (vía API Gateway, con autenticación JWT obligatoria).

Interfaz del servicio: POST `/monitoring/emails` (agregar correo a monitoreo), GET `/monitoring/emails` (listar correos monitorizados del usuario), DELETE `/monitoring/emails/{id}` (eliminar correo del monitoreo), GET `/monitoring/breaches/{email}` (consulta inmediata de brechas para un correo específico).

2.1.3 Servicio de Verificación de Contraseñas

Implementación del servicio: Permite al usuario verificar si una contraseña específica ha sido expuesta en filtraciones conocidas, aplicando el modelo de privacidad k-Anonymity. El proceso es el siguiente: la contraseña se hashea con SHA-1 usando `hashlib` nativo de Python; solo los primeros 5 caracteres del hash (en mayúsculas) se envían a la API HIBP Passwords mediante requests; HIBP retorna todos los hashes que comienzan con esos 5 caracteres; el servidor verifica localmente en memoria si el hash completo aparece en la lista. La contraseña original nunca sale del servidor ni se almacena en ningún momento.

Accesible para todos: Sí (vía API Gateway, con autenticación JWT). La contraseña se transmite solo a través de HTTPS.

Interfaz del servicio: POST `/passwords/check` — Recibe la contraseña en el cuerpo cifrado de la solicitud HTTPS y retorna un objeto JSON con el campo 'count' (número de veces que la contraseña aparece en filtraciones; 0 = no comprometida, >0 = comprometida con el número de apariciones).

2.1.4 Servicio de Notificaciones

Implementación del servicio: Módulo interno invocado directamente por el Módulo de Monitoreo al detectar una brecha. En el MVP local, el módulo imprime la alerta en consola con un formato claro (nombre de la brecha, datos comprometidos, pasos de acción). Opcionalmente puede configurarse `smtplib` con un servidor SMTP de desarrollo local como MailHog para simular el envío de correos sin conexión a internet. No requiere credenciales de servicio externo.

Accesible para todos: No. Es un módulo de uso interno únicamente, invocado directamente desde el Módulo de Monitoreo.

Interfaz del servicio: Función Python: `notify(user_id, email, breach_name, data_classes, breach_date)`. Recibe los datos de la brecha como parámetros y ejecuta la notificación de forma local (consola o SMTP local).

2.1.5 Servicio de Guías de Acción

Implementación del servicio: Genera y retorna guías de acción personalizadas en español según el tipo de datos comprometidos en una brecha. Clasifica las brechas por categoría (contraseñas, datos financieros, correos, números de teléfono, documentos de identidad, etc.) y retorna una guía paso a paso priorizada, incluyendo referencias a servicios colombianos cuando el tipo de brecha lo amerita (p. ej., si hay datos financieros comprometidos, la guía menciona los canales de atención de Bancolombia, Nequi y Davivienda). En el MVP local las guías están definidas en un diccionario Python por tipo de dato comprometido y se renderizan con las plantillas Jinja2 que Flask ya incluye, sin necesidad de base de datos separada para este módulo.

Accesible para todos: Sí (vía API Gateway). También es consumido internamente por el Servicio de Notificaciones para adjuntar la guía en el correo de alerta.

Interfaz del servicio: GET /guides/{breachType} (retorna guía de acción para un tipo de dato comprometido: 'passwords', 'emails', 'financial', 'phone', etc.), GET /guides/breach/{breachName} (retorna guía específica para una brecha conocida por su nombre en HIBP).

2.1.6 Servicio de Historial

Implementación del servicio: Servicio suscriptor del evento 'breach.detected'. Persiste el registro de cada alerta generada por usuario, incluyendo fecha de detección, nombre de la brecha, tipos de datos comprometidos y estado de atención (PENDIENTE / ATENDIDA). En el MVP local utiliza la misma base de datos SQLite del proyecto, con una tabla alerts gestionada con SQLAlchemy. La exportación del historial se realiza en CSV usando el módulo csv nativo de Python.

Accesible para todos: Sí (vía API Gateway, con autenticación JWT). Solo el usuario propietario puede consultar su propio historial.

Interfaz del servicio: GET /history/{userId}?from={fecha}&to={fecha}&page={n} (historial paginado con filtros de fecha), PATCH /history/{alertId}/status (marcar alerta como ATENDIDA), GET /history/{userId}/export?format=pdf|csv (exportación del historial completo).

2.2 Proveedor de servicios

De acuerdo con el principio SOA, el proveedor de servicios es el componente que crea y ofrece un servicio para que sea utilizado por consumidores, sin que estos necesiten conocer los detalles internos de su implementación. En CiberEscudo se identifican dos tipos:

Proveedor externo — Have I Been Pwned (HIBP): Es el proveedor del insumo fundamental del sistema: la base de datos de filtraciones de seguridad a nivel mundial, mantenida por Troy Hunt. Ofrece tres endpoints que CiberEscudo consume: (1) /breachedaccount/{email}: lista de brechas que contienen el correo especificado; (2) /range/{prefix}: verificación de contraseñas por k-Anonymity; (3) /breaches: catálogo

completo de brechas conocidas con metadatos. CiberEscudo actúa como consumidor de este proveedor externo a través del Servicio de Monitoreo y el Servicio de Verificación de Contraseñas, utilizando un módulo adaptador (Adapter Pattern) que abstrae la versión de la API (actualmente v3) del resto del sistema.

Proveedor interno — Plataforma CiberEscudo: La propia plataforma actúa como proveedora de servicios para sus consumidores internos y externos. Los servicios se ofrecen de la siguiente manera: el Servicio de Autenticación provee validación de tokens a todos los demás servicios internos; el Servicio de Guías de Acción provee contenido tanto al Servicio de Notificaciones (para adjuntar guías en correos de alerta) como al cliente web (para mostrar guías en el panel de usuario); el Servicio de Monitoreo provee eventos al bus de mensajes para los suscriptores. Cada servicio proveedor documenta su contrato mediante especificación OpenAPI 3.0 (Swagger), publicada y consultable en el API Gateway, siguiendo el principio SOA de capacidad de descripción del servicio.

La separación entre proveedores externos e internos garantiza que una eventual interrupción o cambio de versión de HIBP solo impacte al módulo adaptador del Servicio de Monitoreo, sin propagarse al resto de los servicios del sistema.

2.3 Registro de los servicios disponibles

El registro de servicios es el componente SOA que centraliza el catálogo de servicios disponibles, permitiendo que los consumidores conozcan los servicios que necesitan invocar. En el MVP local de CiberEscudo se implementa mediante un archivo de configuración Python (config.py) que actúa como registro estático de los módulos del sistema.

El funcionamiento del registro en CiberEscudo es el siguiente:

- En el MVP local, el registro de servicios se implementa mediante un archivo de configuración Python (config.py) que centraliza las URLs de cada módulo en localhost con su puerto asignado (ej. AUTH_URL = 'http://localhost:5001', MONITORING_URL = 'http://localhost:5002'). No se requiere un servidor de registro externo como Consul para el funcionamiento local.
- El archivo config.py centraliza las rutas de importación y los puertos de cada módulo Flask (AUTH: 5001, MONITORING: 5002, PASSWORDS: 5003, GUIDES: 5004, HISTORY: 5005). Cualquier cambio de puerto o ruta se actualiza en un solo lugar.
- Cada módulo Flask expone una ruta /health que retorna {'status': 'ok'}. En el MVP local este endpoint permite verificar manualmente qué módulos están activos antes de ejecutar una sesión de prueba.
- Cuando se despliega una nueva instancia de un servicio (por ejemplo, una segunda instancia del Servicio de Monitoreo durante un pico de carga), esta se

registra automáticamente y el Gateway comienza a distribuir el tráfico entre ambas instancias mediante round-robin, sin ninguna intervención manual.

- El registro también almacena metadatos de configuración de cada servicio: entorno de despliegue (desarrollo / producción), etiquetas de capacidad y la URL de su especificación OpenAPI, funcionando como catálogo de descripción de servicios.

Este mecanismo garantiza que el sistema pueda escalar horizontalmente ante picos de demanda y recuperarse de fallos parciales sin tiempo de inactividad, alineándose con los requerimientos no funcionales de disponibilidad (RNF-03) y escalabilidad (RNF-05) definidos en la Entrega 1.

11. Patrones de arquitectura

11.1 Patrones seleccionados para el sistema

A continuación se presentan los tres patrones de arquitectura seleccionados para CiberEscudo, con su descripción y argumentación respecto al diseño de la aplicación.

6.1 API Gateway

El patrón API Gateway establece un punto de entrada único para todas las solicitudes de los clientes hacia los servicios internos del sistema. En lugar de que el cliente conozca y llame directamente a cada servicio, el Gateway recibe la solicitud, la enruta al servicio correspondiente, aplica políticas transversales y retorna la respuesta consolidada (Richardson, 2018).

Aporte al diseño de CiberEscudo: La aplicación expone múltiples servicios (autenticación, monitoreo, verificación, notificaciones, historial). Sin un API Gateway, el cliente debería conocer la ubicación de cada uno, lo que incrementaría el acoplamiento y la complejidad. Con este patrón, el Gateway centraliza la validación de sesión del usuario, el control de rate limiting —especialmente importante para no exceder los límites de la API HIBP— y el logging de todas las transacciones. Adicionalmente, permite versionar los servicios internos sin exponer cambios al cliente, favoreciendo el cumplimiento del RNF-09.

6.2 Patrón Publicador / Suscriptor (Pub/Sub — Event-Driven)

El patrón Publicador/Suscriptor es un estilo de comunicación asíncrona en el que los componentes productores de eventos (publicadores) no conocen directamente a los componentes que procesarán dichos eventos (suscriptores). La comunicación se realiza a través de un bus de eventos o message broker, como RabbitMQ o Apache Kafka (Richards & Ford, 2020).

Aporte al diseño de CiberEscudo: El proceso de detección de brechas es independiente del proceso de notificación. En el MVP local, cuando el Servicio de Monitoreo detecta una brecha para un correo del usuario, invoca directamente las funciones del Servicio de Notificaciones y del Servicio de Historial. Este desacoplamiento lógico permite que, en una versión futura productiva, se introduzca un bus de mensajes (RabbitMQ) sin rediseñar la lógica de negocio, simplemente reemplazando las llamadas directas por publicación de eventos. Esto contribuye a los RNF-06 (mantenibilidad y modularidad).

6.3 Patrón Repository

El patrón Repository actúa como una capa de abstracción entre la lógica de negocio de los servicios y la capa de persistencia de datos. Los servicios no interactúan directamente con la base de datos, sino que lo hacen a través de interfaces de repositorio que encapsulan las operaciones de acceso a datos (Fowler, 2002).

Aporte al diseño de CiberEscudo: Cada módulo de CiberEscudo (usuarios, correos monitoreados, historial) accede a la base de datos a través de su propia capa de repositorio con SQLAlchemy. En el MVP local se usa SQLite como motor de base de datos, lo que no requiere instalación de servidor. Si en el futuro se requiere mayor capacidad, se puede migrar a PostgreSQL sin modificar la lógica de negocio de cada módulo, ya que el cambio queda contenido en la capa de repositorio.

11.2 Patrones de arquitectura SOA segun Thomas Erl (2009)

A continuación se documentan los tres patrones de arquitectura SOA seleccionados para CiberEscudo, siguiendo el formato propuesto por Erl (2009) en SOA Design Patterns. Para cada patrón se especifica por qué es parte de la solución a la situación problema identificada.

6.1 Patrón: Service Registry (Registro de Servicios)

Característica del patrón SOA Descripción / descripción de la categoría	
Requerimiento	¿Cómo pueden los servicios de CiberEscudo descubrirse mutuamente de forma dinámica, sin acoplamiento directo a ubicaciones fijas de infraestructura?
Problema	En CiberEscudo, el API Gateway y los servicios consumidores (por ejemplo, el Servicio de Notificaciones que invoca al Servicio de Guías de Acción) necesitan conocer la ubicación en red de los demás servicios. Si estas ubicaciones están codificadas directamente en la configuración de cada servicio (IP fija, puerto fijo), cualquier cambio de infraestructura —como escalar el Servicio de Monitoreo ante un evento de brecha masiva— requeriría actualizar manualmente la configuración de todos los consumidores. Esto viola el principio de bajo acoplamiento de SOA y genera fragilidad operativa.
Solución	Se implementa un componente centralizado de Service Registry (Consul) donde cada servicio publica su dirección y estado de salud al iniciarse. Los consumidores consultan el registro para obtener la ubicación actualizada del servicio que necesitan invocar, en lugar de usar direcciones codificadas. El API Gateway actúa como intermediario principal, resolviendo rutas desde el registro antes de cada solicitud.
Aplicación	Cuando el equipo de CiberEscudo despliega una segunda instancia del Servicio de Monitoreo para manejar la carga de una brecha masiva (por ejemplo, la filtración de 50 millones de registros de una plataforma de streaming), esta nueva instancia se registra automáticamente en Consul. El API Gateway detecta las dos instancias disponibles y distribuye el tráfico entre ellas mediante round-robin, sin ningún cambio de configuración manual. Cuando

Característica del patrón SOA	Descripción / descripción de la categoría
	una instancia falla, el health check de Consul la marca como no disponible y el Gateway deja de enviarle solicitudes.
Impacto	Ventajas: Elimina el acoplamiento de ubicación entre servicios. Permite escalar y desplegar servicios independientemente. Facilita la tolerancia a fallos mediante health checks automáticos. Desventajas (trade-offs): El registro en sí mismo es un punto crítico de disponibilidad: si Consul falla, los servicios no pueden descubrirse. Esto se mitiga configurando Consul en modo cluster con múltiples nodos. Añade latencia adicional a cada solicitud por la consulta al registro, lo que requiere implementar caching de resoluciones.
Relaciones	Este patrón es habilitado por el patrón API Gateway (que centraliza el punto de consulta al registro) y es prerequisite del patrón Service Façade. También soporta el patrón Publish-Subscribe al permitir que los consumidores de eventos descubran dinámicamente los endpoints del bus de mensajes.

6.2 Patrón: Publish-Subscribe (Publicador-Suscriptor)

Característica del patrón SOA	Descripción / descripción de la categoría
Requerimiento	¿Cómo puede el Servicio de Monitoreo de CiberEscudo comunicar la detección de una brecha de seguridad a múltiples servicios consumidores (Notificaciones, Historial) sin crear dependencias directas que comprometan la mantenibilidad y disponibilidad del sistema?
Problema	Cuando el Servicio de Monitoreo detecta que el correo de un usuario aparece en una nueva brecha de HIBP, debe desencadenar al menos dos acciones simultáneas: enviar una notificación por correo electrónico (Servicio de Notificaciones) y registrar el evento en el historial del usuario (Servicio de Historial). Si el Monitoreo invoca directamente a cada uno de estos servicios mediante llamadas REST síncronas, se generan las siguientes consecuencias problemáticas: si el Servicio de Notificaciones está caído, el Monitoreo falla aunque el Historial esté disponible; cualquier nuevo

Característica del patrón SOA	Descripción / descripción de la categoría
	servicio que necesite reaccionar a una brecha (por ejemplo, un futuro Servicio de Estadísticas) requeriría modificar el código del Servicio de Monitoreo.
Solución	En el MVP local se implementa una versión simplificada del patrón: el Módulo de Monitoreo, al detectar una brecha, invoca directamente en secuencia las funciones notify() del Módulo de Notificaciones y register_alert() del Módulo de Historial. Aunque son llamadas directas, la lógica está encapsulada en módulos separados con interfaces bien definidas. En una versión productiva futura, estas llamadas directas se reemplazarían por publicación de eventos en RabbitMQ sin modificar la lógica de cada módulo, ya que solo cambia el mecanismo de invocación.
Aplicación	Un usuario registra su correo en CiberEscudo y ejecuta la opción 'verificar ahora'. El Módulo de Monitoreo consulta HIBP con requests, detecta que el correo aparece en una brecha de datos, llama a notify() que imprime la alerta en consola y opcionalmente envía un correo con smtp lib local, y llama a register_alert() que inserta el registro en la base de datos SQLite local. Todo ocurre de forma síncrona y local, sin dependencias externas más allá de la API HIBP.
Impacto	Ventajas: Desacoplamiento total entre el productor y los consumidores del evento. Alta disponibilidad del proceso de detección, independiente del estado de los servicios de notificación e historial. Facilidad para añadir nuevos suscriptores (escalabilidad funcional) sin modificar el publicador. Procesamiento asíncrono que evita bloqueos. Desventajas (trade-offs): Se introduce consistencia eventual: el usuario no es notificado en el instante exacto de la detección, sino cuando el suscriptor procesa el mensaje. Requiere monitoreo del backlog de colas para detectar acumulaciones. La depuración de flujos de eventos es más compleja que en sistemas síncronos.
Relaciones	Depende del patrón Service Registry para que los suscriptores descubran la dirección del broker. Es habilitado por el patrón Service Façade (API Gateway), que puede actuar como punto de entrada para eventos externos. Complementa el patrón Service Registry al

Característica del patrón SOA	Descripción / descripción de la categoría
	permitir comunicación entre servicios sin necesidad de conocer sus ubicaciones directamente.

6.3 Patrón: Service Façade (Fachada de Servicio / API Gateway)

Característica del patrón SOA	Descripción / descripción de la categoría
Requerimiento	¿Cómo puede el cliente web de CiberEscudo interactuar con múltiples servicios internos mediante una interfaz simplificada, sin exponerse a la complejidad interna del sistema ni verse afectado por los cambios en la estructura de servicios?
Problema	El frontend web de CiberEscudo necesita, para renderizar el dashboard del usuario, información proveniente de al menos tres servicios: el Servicio de Monitoreo (brechas actuales), el Servicio de Historial (alertas anteriores) y el Servicio de Autenticación (validación del token). Sin una fachada, el cliente tendería que: conocer las URL individuales de cada servicio, gestionar múltiples tokens de autenticación, manejar versiones diferentes de cada API, y realizar llamadas paralelas y consolidar respuestas. Esto acopla el cliente a la estructura interna del sistema y dificulta cualquier refactorización de servicios.
Solución	Se implementa una Fachada de Servicio materializada como API Gateway (utilizando Kong o AWS API Gateway). Esta fachada expone una API unificada al cliente, internamente orquestando las llamadas a los servicios necesarios, aplicando políticas transversales (autenticación JWT, rate limiting, logging, CORS) y consolidando respuestas. El cliente interactúa exclusivamente con el Gateway, ignorando completamente la topología interna.
Aplicación	Cuando el usuario de CiberEscudo inicia sesión y accede a su dashboard, el frontend realiza una sola petición GET /dashboard a la fachada. El API Gateway: valida el token JWT, consulta al Servicio de Autenticación para verificar permisos, invoca en paralelo al Servicio de Monitoreo (brechas del usuario) y al Servicio de Historial (últimas 5 alertas), consolida las respuestas en un único objeto JSON y lo retorna al cliente en una sola respuesta. Si en el futuro el

Característica del patrón SOA	Descripción / descripción de la categoría
	equipo decide separar el Servicio de Historial en dos (uno para alertas, uno para estadísticas), el cliente no percibe el cambio: el Gateway absorbe la diferencia.
Impacto	Ventajas: Simplifica el cliente al ocultar la complejidad interna del sistema. Centraliza políticas transversales (seguridad, logging, throttling) evitando duplicación en cada servicio. Facilita el versionado de la API sin impactar el cliente. Permite refactorizar servicios internos sin cambios en el contrato externo. Desventajas (trade-offs): El API Gateway puede convertirse en un cuello de botella si no se escala horizontalmente. Añade un salto de red adicional a cada solicitud. Concentra la complejidad de orquestación en un único componente que debe ser altamente confiable.
Relaciones	Es el patrón central de la arquitectura de CiberEscudo. Depende del patrón Service Registry para descubrir los servicios internos que debe orquestar. Habilita el patrón Publish-Subscribe al poder actuar como punto de entrada para webhooks externos (por ejemplo, notificaciones de nuevas brechas desde HIBP). Los tres patrones forman un conjunto cohesivo: el Registry permite el descubrimiento, la Fachada centraliza el acceso, y el Pub-Sub gestiona la comunicación asíncrona interna.

ENTREGA FINAL

SEMANA 7

12. Protocolo de comunicacion: REST vs SOAP en CiberEscudo

Teniendo en cuenta el patrón de arquitectura SOA definido para CiberEscudo y las características del sistema, se procede a analizar los dos protocolos estándar para la exposición de servicios — REST y SOAP — con el fin de argumentar cuál de ellos incide de manera más positiva en la transformación e implementación de la arquitectura propuesta.

7.1 Análisis comparativo

Criterio	REST	SOAP
Formato de datos	JSON (liviano; consumido de forma nativa en Python con la librería json estándar y Flask jsonify)	XML (verboso, mayor sobrecarga de red)
Protocolo base	HTTP/HTTPS con uso semántico de verbos (GET, POST, PUT, DELETE)	HTTP, SMTP, TCP — independiente del protocolo, pero más complejo
Estado (Stateless)	Stateless por diseño, ideal para escalado horizontal	Puede ser stateful, lo que dificulta el escalado
Compatibilidad con HIBP	Compatibilidad nativa: HIBP es una API REST	Requeriría una capa de adaptación adicional
Compatibilidad con clientes web/móvil	Soporte nativo: Python consume REST con httpx/requests; el frontend web lo consume con fetch/axios de forma directa	Requiere librerías SOAP adicionales, más complejo en frontend
Seguridad	HTTPS + JWT. Implementado sobre TLS estándar	WS-Security: más robusto para transacciones empresariales críticas
Contrato de servicio	OpenAPI 3.0 / Swagger (flexible y autodocumentado)	WSDL (estricto y detallado, pero más rígido)

Criterio	REST	SOAP
Curva de aprendizaje	Baja: ampliamente conocido por el equipo de desarrollo	Alta: requiere conocimiento especializado de estándares WS-*
Rendimiento	Alto: payloads JSON son hasta 10x más pequeños que XML equivalente	Menor: overhead de XML y envelope SOAP impacta la latencia
Casos de uso ideales	Aplicaciones web, móviles, APIs públicas, microservicios	Integración empresarial B2B, transacciones bancarias con ACID garantizado

7.2 Protocolo seleccionado: REST

CiberEscudo adopta REST como protocolo estándar para la exposición de todos sus servicios, por las siguientes razones que inciden directamente en la transformación de la implementación arquitectónica:

Compatibilidad con el ecosistema de servicios externos: La API HIBP es una API REST que retorna respuestas en JSON. Adoptar REST en los servicios internos de CiberEscudo permite que el Servicio de Monitoreo y el Servicio de Verificación de Contraseñas consuman HIBP sin capas de adaptación adicionales, reduciendo la complejidad del sistema y los puntos de fallo.

Alineación con el perfil tecnológico del equipo y del cliente: El backend del MVP de CiberEscudo está desarrollado en Python con Flask, que retorna JSON de forma nativa mediante `jsonify()` sin configuración adicional. La interfaz web del MVP usa plantillas Jinja2 renderizadas por Flask (sin framework de frontend separado), lo que elimina la necesidad de gestionar una capa de API independiente para el cliente. SOAP añadiría complejidad de serialización XML innecesaria para un proyecto local de esta escala.

Soporte al escalado horizontal (RNF-05): REST es stateless por diseño: cada solicitud contiene toda la información necesaria para ser procesada. Esto permite que el API Gateway distribuya solicitudes entre múltiples instancias de un servicio sin sesiones compartidas, alineándose directamente con el requerimiento de soportar al menos 10.000 usuarios concurrentes.

Documentación y contrato de servicio con OpenAPI 3.0: Cada servicio de CiberEscudo documenta su interfaz mediante Swagger UI, generando documentación interactiva automáticamente. Esto facilita la integración entre servicios internos y reduce

la fricción en el desarrollo. En SOAP, el equivalente (WSDL) es significativamente más complejo de generar y mantener.

Rendimiento bajo condiciones de alta carga: Durante un evento de brecha masiva, el Servicio de Monitoreo puede procesar miles de consultas por minuto. Un payload JSON de respuesta de HIBP ocupa en promedio 1-3 KB, mientras que el equivalente en XML/SOAP superaría los 8 KB. En escenarios de alta concurrencia, esta diferencia impacta significativamente el consumo de ancho de banda y el tiempo de respuesta (RNF-04).

La única consideración donde SOAP presentaría ventaja sería en integraciones B2B con instituciones financieras que exigen contratos WS-Security y garantías ACID. Para este caso puntual, CiberEscudo podría implementar un adaptador SOAP en el futuro sin modificar su arquitectura REST interna.

13. Propuesta de servicio REST

Teniendo en cuenta la arquitectura SOA planteada se propone implementar un conjunto de servicios bajo el estilo arquitectónico REST debido a su flexibilidad, bajo consumo de recursos, facilidad de integración con aplicaciones web y compatibilidad con protocolos estándar HTTP. REST es la mejor opción ya que permite la comunicación eficiente entre el cliente, el API Gateway y los servicios internos del sistema, usando repuestas en formatos JSON.

Endpoints del servicio de autenticación

POST/auth/login

Se usa para iniciar sesión.

```
{  
  "usuario": "admin",  
  "password": "123456"  
}
```

Si son correctos el sistema responde.

```
{  
  "mensaje": "Acceso autorizado",  
  "token": "jwt_token_aqui"  
}
```

POST/auth/logout

Permite cerrar sesión del usuario.

```
{  
  "mensaje": "Sesión cerrada correctamente"  
}
```

GET/auth/profile

Permite consultar la información del usuario autenticado.

```
{  
  "id": 1,  
  "nombre": "Yeison Hernandez",  
  "correo": "yeison@email.com"  
}
```

14. Guía de instalación y ejecución local

Esta sección describe los pasos necesarios para clonar el repositorio, configurar el entorno virtual, instalar las dependencias e iniciar la aplicación CiberEscudo en un equipo local.

5.1 Requisitos previos

Python 3.10 o superior — <https://www.python.org/downloads/>

Git — <https://git-scm.com/>

Terminal (PowerShell, CMD o bash en macOS/Linux)

5.2 Clonar el repositorio

Ejecuta los siguientes comandos en tu terminal:

```
git clone https://github.com/crlc4rd0/POLI_ARQUITECTURA_DE_SOFTWARE.git  
cd POLI_ARQUITECTURA_DE_SOFTWARE
```

5.3 Crear el entorno virtual

Se recomienda usar un entorno virtual para aislar las dependencias del proyecto y evitar conflictos con otras instalaciones de Python en el equipo.

En Windows:

```
python -m venv .venv  
  
.venv\Scripts\activate
```

En macOS / Linux:

```
python3 -m venv .venv  
  
source .venv/bin/activate
```

Cuando el entorno este activo, el prompt de la terminal mostrara el prefijo (.venv) al inicio de la linea.

5.4 Instalar Flask y dependencias

Con el entorno virtual activo, instala todos los paquetes necesarios:

```
pip install -r requirements.txt
```

El archivo requirements.txt incluye los siguientes paquetes:

flask 3.1.3 — Framework web principal.

requests 2.32.3 — Consultas HTTP a la API de Have I Been Pwned.

python-dotenv 1.0.1 — Carga de variables de entorno desde archivo .env.

5.5 Ejecutar el proyecto

Desde la raiz del repositorio, ejecuta:

```
python app.py
```

Deberias ver en consola una salida similar a la siguiente:

```
* Running on http://127.0.0.1:5000  
  
* Debug mode: on
```

Abre tu navegador y navega a <http://127.0.0.1:5000> para acceder a la aplicacion.

5.6 Iniciar sesion

CiberEscudo utiliza un usuario fijo para el entorno de desarrollo (MVP local). No es necesario registrarse. Las credenciales de acceso son:

Usuario: admin

Contraseña: ciberescudo123

Ingresa las credenciales en la pantalla de login y haz clic en Iniciar Sesión. El sistema validará las credenciales y redirigirá automáticamente al panel principal de CiberEscudo.

5.7 Estructura del proyecto

Organización de carpetas y archivos principales del proyecto:

app.py	- Punto de entrada principal de Flask.
config.py	- Configuración global y registro de servicios (SOA).
requirements.txt	- Lista de dependencias Python.
routes/	- Blueprints de cada servicio SOA.
auth.py	- Servicio de autenticación.
monitoring.py	- Servicio de monitoreo de correos.
passwords.py	- Servicio de verificación de contraseñas.
guides.py	- Servicio de guías de acción.
history.py	- Servicio de historial de alertas.
services/	- Lógica de negocio de cada servicio.
hibp_service.py	- Adaptador HIBP (brechas de correo).
password_checker.py	- Verificación con k-Anonymity.
action_guides.py	- Guías en español con referencias colombianas.
history_service.py	- Persistencia SQLite.
templates/	- Vistas HTML con Jinja2 y Bootstrap 5.
static/css/	- Estilos personalizados del tema oscuro.

5.8 Endpoints disponibles

Rutas HTTP expuestas por la aplicación:

GET / Panel principal (requiere sesión activa).

GET/POST /auth/login Formulario de inicio de sesión.

GET /auth/logout Cierre de sesión.

POST /monitoring/check Verificar si un correo fue filtrado.

GET/POST /passwords/check Verificar si una contraseña fue comprometida.

GET /history/ Historial de alertas registradas.

GET /guides/<tipo> Guía de acción para un tipo de dato expuesto.

GET /api/v1/health Estado global del sistema.

15. Conclusiones

A partir del análisis desarrollado en las tres entregas del proyecto CiberEscudo, se derivan las siguientes conclusiones:

- La situación problema identificada —la falta de visibilidad, detección oportuna y orientación post-brecha para el ciudadano colombiano en materia de seguridad digital— es una problemática real, recurrente y de alta relevancia social. Su análisis desde la perspectiva de la arquitectura de software orientada a servicios permite evidenciar cómo SOA proporciona un marco estructural adecuado para abordar problemas complejos que involucran múltiples actores, fuentes de datos externas y requisitos de disponibilidad críticos.
- El análisis de los componentes SOA de CiberEscudo (servicios, proveedores y registro) demuestra que la descomposición del sistema en seis servicios independientes —Autenticación, Monitoreo, Verificación de Contraseñas, Notificaciones, Guías de Acción e Historial— permite satisfacer simultáneamente los atributos de calidad más críticos: alta disponibilidad, escalabilidad horizontal, mantenibilidad independiente y cumplimiento normativo. Cada servicio tiene una responsabilidad única y contratos de interfaz bien definidos, lo que facilita su evolución sin afectar al resto del sistema.
- Los tres patrones SOA de Thomas Erl (2009) seleccionados —Service Registry, Publish-Subscribe y Service Façade— son complementarios y forman un conjunto arquitectónico coherente. El Registry habilita el descubrimiento dinámico, la Fachada centraliza el acceso y simplifica el cliente, y el Pub-Sub garantiza la comunicación asíncrona tolerante a fallos entre servicios internos. Su combinación permite que el sistema responda a la situación problema con robustez operativa y bajo acoplamiento estructural.
- La selección de REST como protocolo de comunicación para CiberEscudo es la decisión técnica correcta en el contexto del problema. La compatibilidad nativa con la API HIBP, la ligereza del formato JSON frente a XML/SOAP, el

soporte al escalado stateless y la familiaridad del equipo con el ecosistema REST/HTTP son factores determinantes. Esta elección transforma positivamente la implementación de la arquitectura al eliminar capas de adaptación innecesarias y reducir la latencia bajo condiciones de alta carga.

- El presente análisis evidencia que la arquitectura de software no es un artefacto estático sino un proceso iterativo de toma de decisiones fundamentadas. Cada decisión —el estilo SOA, los patrones seleccionados, el protocolo REST— responde a atributos de calidad concretos (RNF) y a las restricciones del problema real. Este es el valor central de la arquitectura de software como disciplina: proveer un mapa de decisiones que guíe el desarrollo hacia sistemas robustos, escalables y mantenibles, antes de escribir una sola línea de código.

15.1 Reflexiones finales sobre la arquitectura

¿Se respondió al objetivo planteado de acuerdo con el patrón seleccionado?

Sí, el objetivo planteado fue alcanzado de manera satisfactoria mediante la implementación de una arquitectura orientada a servicios. El uso de los patrones seleccionados permitió estructurar CiberEscudo como un sistema organizado, modular y eficiente, capaz de consultar filtraciones de datos, generar alertas y brindar apoyo al usuario para proteger su información.

¿Qué aspectos incluiría en el análisis para justificar otros patrones de arquitectura?

Para seleccionar otros patrones arquitectónicos sería importante analizar factores como la cantidad de usuarios que utilizarán la plataforma, el crecimiento esperado del sistema, la velocidad de respuesta requerida, el nivel de seguridad necesario, la facilidad de mantenimiento y los recursos disponibles. Con base en estos criterios, podrían evaluarse alternativas como microservicios, arquitectura en capas o modelos basados en eventos.

¿Qué ventajas trae la implementación de arquitecturas orientadas a servicios como REST o SOAP?

Tecnología	Ventajas	Desventajas
REST	Es rápido y ligero en el intercambio de datos. Usa formatos simples como JSON. Fácil integración con aplicaciones web y móviles. Menor consumo de recursos. Flexible y escalable.	Menor nivel de seguridad nativo. No maneja estándares estrictos. Puede requerir configuraciones adicionales para transacciones complejas.
SOAP	Alta seguridad mediante estándares como WS-Security. Mayor confiabilidad en procesos empresariales. Soporta transacciones complejas. Contratos formales mediante WSDL.	Más pesado y lento por usar XML. Mayor consumo de recursos. Implementación más compleja. Menor flexibilidad frente a REST.

Después del análisis realizado, se concluye que REST es la alternativa más adecuada para la implementación de los servicios en CiberEscudo, debido a que ofrece una comunicación rápida, sencilla y eficiente entre los diferentes componentes del sistema. Su uso de protocolos estándar como HTTP y formatos ligeros como JSON facilita la integración con aplicaciones web modernas y con servicios externos.

Además, REST permite desarrollar una arquitectura escalable y flexible, donde cada servicio puede evolucionar de manera independiente sin afectar el funcionamiento general de la plataforma. Esto mejora el mantenimiento del sistema, reduce costos de desarrollo y optimiza el rendimiento.

En este sentido, la adopción de REST fortalece la capacidad de CiberEscudo para brindar una solución moderna, accesible y preparada para responder a futuras necesidades tecnológicas y de seguridad informática.

16. Referencias

Amazon Web Services. (2022). ¿Qué es la arquitectura orientada a servicios? <https://aws.amazon.com/es/what-is/service-oriented-architecture/>

Caivano, D., Fernández-Alemán, J. L., & Toval, A. (2019). Personal data protection in social networking sites: A systematic review. *Computers & Security*, 77, 382-400. <https://doi.org/10.1016/j.cose.2018.05.004>

Erl, T. (2009). *SOA design patterns*. Prentice Hall.

Hunt, T. (2018). Have I Been Pwned: Passwords API v3 documentation. <https://haveibeenpwned.com/API/v3>

Huet, P. (2022). Arquitectura de software: qué es y qué tipos existen. <https://openwebinars.net/blog/arquitectura-de-software-que-es-y-que-tipos-existen/>

IBM Security. (2023). Cost of a data breach report 2023. IBM Corporation. <https://www.ibm.com/reports/data-breach>

OWASP Foundation. (2021). OWASP top ten 2021. <https://owasp.org/www-project-top-ten/>

Richards, M., & Ford, N. (2020). *Fundamentals of software architecture: An engineering approach*. O'Reilly Media.

Richardson, C. (2018). *Microservices patterns: With examples in Java*. Manning Publications.

Shostack, A. (2014). *Threat modeling: Designing for security*. Wiley.

Superintendencia de Industria y Comercio. (2022). Informe de actividad de datos personales en Colombia 2022. <https://www.sic.gov.co/>

HashiCorp. (2023). Consul documentation: Service discovery. <https://developer.hashicorp.com/consul/docs/concepts/service-discovery>